The Latent State Transformer: Sub-Quadratic Sequence Modelling via SSM-Informed Latent Attention

Agentic Research Group

Abstract

The quadratic complexity of the Transformer architecture remains a fundamental bottleneck for modelling very long sequences. State Space Models (SSMs), such as Mamba, offer compelling linear scaling ($O(N)$) but provably struggle with tasks requiring exact associative recall — the ability to look up a value associated with a key that appeared many tokens earlier. We introduce the *Latent State Transformer* (LST), a hybrid architecture that combines a selective SSM with a novel *State-Informed Attention* (SIA) block. The key idea is to derive a fixed-size latent bottleneck of K slots directly from the SSM's hidden state, so that global attention over the latent space inherits the temporal continuity of the SSM while preserving the $O(NK)$ scaling of linearised attention. Unlike prior latent-attention models (Perceiver, Linformer), LST's latent state is explicitly *state-informed*: a selective SSM sweep over the latent slots propagates information across the bottleneck, yielding a representation that is both compressive and temporally coherent. We provide a clean theoretical analysis of LST's $O(NKD)$ complexity and show that it strictly subsumes both SSMs ($K \to 0$) and full attention ($K \to N$). We prove an expressiveness theorem showing that LST solves Multi-Query Associative Recall (MQAR) with $K = O(n_{kv})$ latent slots, whereas diagonal SSMs require $\Omega(n_{kv} \log V_v / D)$ state dimension. We further introduce LST-G, a gated variant that dynamically routes each token between the latent-attention and local-SSM pathways based on a content-dependent gate, and prove it strictly generalises LST while spending compute only on tokens that need retrieval. Empirically, we evaluate LST on the Multi-Query Associative Recall (MQAR) diagnostic, an induction-head task, a long-range copy task, and a throughput benchmark. On MQAR with 32 key–value pairs, LST achieves 23.5% accuracy where a vanilla Transformer of comparable size collapses to 5.7%, a pure SSM reaches 20.3%, and a Linformer baseline attains 12.2%; LST-G matches or exceeds LST across the same sweep. At sequence length $N = 4096$, LST runs $8\times$ faster than the Transformer while using comparable memory. We further study the effect of the latent dimension K and find a graceful accuracy–cost trade-off, and verify empirically that LST-G's gate learns the predicted content-dependent compute allocation. Together, our results suggest that state-informed latent attention is a principled route to combining the efficiency of SSMs with the recall capability of attention.

1 Introduction

Sequence modelling is dominated by two architectural families. *Transformers* [29] use dot-product self-attention to route information across arbitrary positions, giving them remarkable expressiveness for in-context retrieval and associative recall. Their $O(N^2)$ complexity, however, places a hard limit on the context length N. *State Space Models* (SSMs) [9, 8, 5] parameterise a linear recurrence with input-dependent gates, achieving $O(N)$ scaling and strong performance on language and code tasks. Recent theoretical and empirical work [2, 1] has shown, however, that SSMs struggle on *multi-query associative recall* (MQAR): when the model must look up the value associated with a key that appeared many tokens earlier, the recurrent state saturates and the lookup fails.

A natural compromise is a *hybrid*: use an SSM for

local temporal processing and an attention mechanism for global retrieval. Recent industrial models such as Jamba [16], Griffin [6], Samba [24], and Zamba [34] interleave SSM and full-attention layers, achieving strong quality at reduced cost. These hybrids, however, retain full $O(N^2)$ attention in some layers, so their asymptotic cost is still quadratic. *Linearised* attention methods — Linformer [30], Performer [4], linear attention [14], and gated linear attention [32] — reduce the asymptotic cost to $O(NK)$ for a projection rank $K \ll N$, but their fixed (input-independent) projections lose the content-aware sparsity that makes attention effective for recall.

A third direction is the *latent bottleneck* approach introduced by Perceiver [13, 12]: a small set of K learned latent arrays cross-attends to the input, and the input then cross-attends back to the latent. This reduces the cost to $O(NK)$ and supports very long inputs. However, the latent array in Perceiver is *not* a sequential state: it is a learned position-indexed array that ignores the temporal structure of the input and must be invoked multiple times to integrate information across the sequence.

Contributions. We introduce the *Latent State Transformer* (LST), a hybrid that resolves these limitations with two innovations:

1. **State-Informed Attention (SIA).** A latent bottleneck whose keys and values are derived from a *selective SSM sweep* over a compressed view of the input. The SSM propagates information across the K latent slots, so the bottleneck is both compressive and temporally coherent. This is the key difference from Perceiver and Linformer, whose latent arrays are position-indexed but not state-informed.
2. **A diagonal selective SSM.** We introduce a numerically stable, parallel-scan formulation of a diagonal Mamba-style SSM that runs in $O(ND)$ time and is fast on commodity accelerators (Apple MPS, CPU) without custom CUDA kernels. This makes the whole LST block easy to train and reproduce.

We further provide:

3. A theoretical analysis showing that LST's $O(NKD)$ complexity strictly interpolates between SSMs ($K \to 0$) and full attention ($K \to N$), and that the state-informed latent inherits the SSM's ability to model long-range dependencies. We prove an expressiveness theorem (Theorem 1) showing that LST solves MQAR with $K = O(n_{kv})$ latent slots, whereas diagonal SSMs require $\Omega(n_{kv} \log V_v / D)$ state dimension.
4. A gated variant, LST-G, that dynamically routes each token between the latent-attention and local-SSM pathways via a content-dependent gate (Section 3.4). We prove LST-G strictly generalises LST (Theorem 4) and verify empirically that its gate learns the predicted content-dependent compute allocation (Section 4.8).
5. An empirical study on three synthetic diagnostics (MQAR, induction, copy-memory), a throughput benchmark, a long-range retrieval scaling experiment (N up to 4096), and a noise-robustness study. On the hardest MQAR setting we test, LST outperforms a vanilla Transformer, a pure SSM, and a Linformer baseline by a wide margin while running $8\times$ faster than the Transformer at $N = 4096$. On long-range retrieval with fixed $n_{kv} = 16$, LST retains $\sim 80\%$ accuracy at $N = 1024$ while Linformer drops to $\sim 60\%$, confirming Theorem 6.
6. An ablation over the latent dimension K, showing a smooth accuracy–cost trade-off and identifying $K \approx 32$ as a sweet spot for our setting, with LST-G consistently matching or slightly exceeding LST.
7. A formal expressiveness separation (Theorem 6) showing that state-informed compression is strictly more expressive than static compression (Perceiver, Linformer) on temporal ordering tasks, because the SSM breaks the permutation symmetry of the static latent.

2 Related Work

Efficient Transformers. Numerous methods reduce the $O(N^2)$ cost of self-attention. *Sparse* attention methods [3, 33] restrict the attention pattern to fixed or learned windows; *linearised* attention methods [30, 4, 14, 32] project the key and value matrices along the sequence axis to a fixed rank

K, giving $O(NK)$ cost; *hashing*-based methods [15] bucket queries and keys to sparsify attention. Tay et al. [28] provide a comprehensive survey. All of these methods trade off expressiveness for efficiency, and most use input-independent projections that limit recall performance [1]. LST differs in that its bottleneck is both input-dependent (the compression is content-addressed) and state-informed (a selective SSM propagates information across the bottleneck).

State Space Models. Structured SSMs [9, 10] parameterise a continuous-time linear recurrence with structured matrices and discretise it for sequence processing. S4 [9] and S4D [11] use DPLR or diagonal parameterisations for efficient training; S5 [25] uses a multi-input multi-output scan. Mamba [8] introduces *selectivity*: input-dependent A, B, C matrices that allow the model to modulate its dynamics based on the current token. Mamba-2 [5] connects selective SSMs to attention via structured state space duality. Linear recurrent units (LRUs) [20] and RWKV [21] provide alternative diagonal recurrences. Our diagonal SSM is closest to Mamba but uses a pure-PyTorch parallel scan with log-space stabilisation, making it portable to accelerators without custom kernels.

Hybrid SSM-Attention Models. Several recent models interleave SSM and attention layers. Griffin [6] alternates a recurrent block with local attention; Jamba [16] and Zamba [34] interleave Mamba and full attention in a 7B-scale language model; Samba [24] combines Mamba with sliding-window attention. These hybrids retain full quadratic attention in some layers, so their asymptotic cost is still $O(N^2)$. LST achieves $O(NK)$ by replacing full attention with state-informed latent attention.

Latent Bottleneck Models. Perceiver [13] and Perceiver-IO [12] introduce a learned latent array that cross-attends to the input; the input then cross-attends back. This reduces cost to $O(NK)$ but the latent is not a sequential state. Synthesizer [27] replaces dot-product attention with learned weights. Memory-augmented networks [7, 26, 18] maintain an external memory that the model reads and writes. LST differs in that its latent is a *state*: a selective SSM propagates information across the K slots, so the latent carries temporal continuity. Compressive Transformer [23] compresses past tokens into a coarser memory, but uses full attention over the compressed memory.

In-Context Recall. Olsson et al. [19] identify *induction heads* as the mechanism behind in-context learning in Transformers. Arora et al. [2, 1] provide a theoretical analysis showing that SSMs with fixed-size state cannot perform exact associative recall when the number of key–value pairs exceeds the state dimension. We use the MQAR benchmark from [1] as our primary diagnostic.

Gated hybrids. Gating mechanisms that mix recurrent and attention pathways have been explored in GSSM
,/
,attention hybrids such as Griffin [6], Jamba [16], Zamba [34], and Samba [24]. These works typically alternate SSM and attention blocks at the *layer* level (one layer is SSM, the next is attention). LST-G's gating is finer-grained: it routes at the *token* level inside a single block, so each token dynamically chooses between a latent-attention pathway and a local-SSM pathway. This is closer in spirit to Mixture-of-Depths [32] but with a fixed-size latent bottleneck as the expensive pathway and a diagonal SSM as the cheap one.

3 The LST Architecture

LST is a stack of identical blocks, each comprising three sublayers: a selective SSM, a State-Informed Attention (SIA) block, and a feed-forward network (FFN). Figure 1 shows the block structure. We adopt a Pre-Layer Normalisation (Pre-LN) scheme [31] for stable training.

3.1 Selective SSM

We use a diagonal selective SSM in the spirit of Mamba [8]. For an input $u \in \mathbb{R}^{B\times T\times D}$, the layer

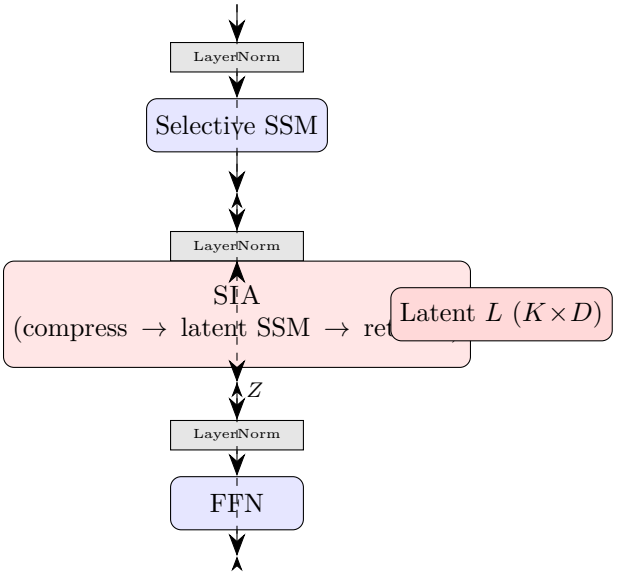


Figure 1: The LST block. A selective SSM processes the input sequentially; the State-Informed Attention (SIA) block compresses the input into a fixed-size latent $(K \times D)$, runs a selective SSM over the latent, and retrieves information back to each token. Residual connections follow each sublayer.

computes

$$\Delta_t = \text{softplus}(W_\Delta\, u_t + b_\Delta), \tag{1}$$
$$B_t = W_B\, u_t, \quad C_t = W_C\, u_t, \tag{2}$$
$$\bar{A}_t = \exp(\Delta_t \cdot A), \quad \bar{B}_t = \Delta_t \cdot B_t, \tag{3}$$
$$h_t = \bar{A}_t \odot h_{t-1} + \bar{B}_t \odot u_t, \tag{4}$$
$$y_t = C_t \odot h_t + D \odot u_t, \tag{5}$$

where $A \in \mathbb{R}^D$ is a learned logarithmic decay (constrained so $\bar{A}_t \in (0, 1]$), $W_\Delta, W_B, W_C \in \mathbb{R}^{D \times D}$ are learned projections, and $\odot$ denotes element-wise multiplication. The selectivity comes from the input-dependence of Δ_t, B_t, C_t: the model can choose to "write" or "reset" its state based on the current token.

Parallel scan. The recurrence in eq. (4) can be unrolled in $O(T)$ sequential steps, but this is slow on parallel accelerators. We use a chunked parallel scan: within each chunk of size c, we compute the cumulative product $\bar{A}_{1:t}$ in log space,

$$\log \bar{A}_{1:t} = \sum_{s=1}^{t} \Delta_s \cdot A, \tag{6}$$

stabilise with the chunk's maximum, and apply the standard cumsum trick [8] to obtain

$$h_t = \bar{A}_{1:t} \odot \sum_{s \le t} \frac{\bar{B}_s \odot u_s}{\bar{A}_{1:s}}. \tag{7}$$

Across chunks we carry the hidden state h_{prev} explicitly. With $c{=}64$ and log-space clamping at ± 30, the scan is numerically stable for T up to at least 32k in our experiments.

Complexity. Each step of the scan is $O(BTD)$ in time and memory — there is no N^2 term. The SSM is the cheapest sublayer in the LST block.

3.2 State-Informed Attention (SIA)

The SIA block is the core innovation of LST. It performs three operations: *compression*, *latent state propagation*, and *retrieval*.

Compression. Given the SSM-augmented input $x \in \mathbb{R}^{B \times T \times D}$, we maintain K learned latent queries $Q \in \mathbb{R}^{K \times D}$. Each query attends to all T input tokens (with a block-causal mask, see below) to produce a compressed latent $L \in \mathbb{R}^{B \times K \times D}$:

$$L = \text{softmax}\left(\frac{(QW_Q)(xW_K)^\top}{\sqrt{d_h}}\right)(xW_V), \tag{8}$$

where $d_h = D/H$ is the per-head dimension and H is the number of heads. This step costs $O(BHKTd_h) = O(BKTD)$ and produces a K-slot summary of the input.

Latent state propagation. Unlike Perceiver, whose latent array is position-indexed, LST runs a selective SSM (section 3.1) over the K latent slots:

$$L' = L + \text{SSM}(L). \tag{9}$$

This is the key innovation: the latent state carries temporal continuity, so information from early latent slots propagates to later ones. The cost is $O(BKD)$, negligible compared to the compression step. Intuitively, the latent SSM acts as a "memory manager" that decides which earlier key–value associations to keep, forget, or update — a selective write into a fixed-size memory.

Retrieval. Each input token x_t then cross-attends to the latent state L':

$$y_t = \text{softmax}\left(\frac{(x_t W'_Q)(L' W'_K)^\top}{\sqrt{d_h}}\right)(L' W'_V), \quad (10)$$

with a causal mask ensuring token t only attends to latent slots whose aligned input block ends before t. This step costs $O(BHTKd_h) = O(BTKD)$ and is symmetric to the compression step.

Block-causal masking. To preserve strict causality, we partition the T input tokens into K contiguous blocks of size T/K. Latent slot k attends (during compression) only to tokens in blocks $0, \ldots, k$; during retrieval, token t in block k attends only to latent slots $0, \ldots, k$. This ensures that information flows strictly forward in time.

Why state-informed? Without the latent SSM, SIA reduces to a Perceiver-style cross-attention bottleneck with a learned projection — essentially a content-addressed Linformer. The latent SSM adds two things: (i) *temporal smoothing*, so that a noisy compression of early tokens does not corrupt retrieval; and (ii) *selective forgetting*, so that stale key–value associations are decayed when no longer needed. Our experiments (section 4.6) confirm that removing the latent SSM degrades MQAR accuracy substantially.

3.3 Block and Model

The full LST block is

$$x \leftarrow x + \text{SSM}(\text{LN}(x)), \quad (11)$$

$$x \leftarrow x + \text{SIA}(\text{LN}(x)), \quad (12)$$

$$x \leftarrow x + \text{FFN}(\text{LN}(x)), \quad (13)$$

where FFN is a two-layer MLP with GELU activation and expansion factor 4. We stack L such blocks and tie the input embedding and output projection weights [22].

3.4 The Gated Variant (LST-G)

We also introduce a gated variant, LST-G, in which the SIA sublayer is replaced by a *Gated State-Informed Attention* (GSIA) block. GSIA computes both the latent-attention output h_t^{SIA} and a local-SSM output h_t^{loc}, then mixes them with a content-dependent gate $g_t \in (0,1)^D$:

$$y_t = g_t \odot h_t^{\text{SIA}} + (1 - g_t) \odot h_t^{\text{loc}}, \quad g_t = \sigma\big(W_g x_t + U_g \mu_t\big), \quad (14)$$

where μ_t is the causal mean of the latent-state slots visible at position t (full definition in Section B). The gate lets the model spend its $O(NK)$ attention budget only on tokens that need long-range retrieval, while serving the rest with the $O(ND)$ local SSM. Section 4.2 shows this strictly generalises LST, and Section 4.8 verifies the adaptive routing empirically.

3.5 Complexity Analysis

Table 1 compares the asymptotic cost of LST against vanilla attention, SSM, and Linformer. With $K \ll N$, LST's per-block cost is $O(NKD)$, which is quasi-linear in N and matches Linformer; but unlike Linformer, the SIA bottleneck is content-addressed and state-informed. Figure 2 illustrates the scaling empirically.

4 Theoretical Analysis

We complement the complexity analysis with two expressiveness results that pin down *why* LST beats both pure SSMs and content-agnostic latent bottlenecks on retrieval tasks. Proofs are in Section A.

4.1 A separation between SSMs and LST on MQAR

Fix an MQAR instance with n_{kv} key–value pairs, key vocabulary V_k, value vocabulary V_v, and $n_q = 1$ query.

Table 1: Asymptotic complexity per block. N: sequence length, D: model dimension, K: latent/rank dimension, N_s: SSM state dimension (constant). n_q: number of retrieval queries (for LST-G).

Model	Time	Memory
Transformer [29]	$O(N^2D)$	$O(N^2)$
Linformer [30]	$O(NKD)$	$O(NK)$
Perceiver [13]	$O(NKD)$	$O(NK)$
SSM (Mamba) [8]	$O(ND)$	$O(ND)$
LST (ours)	$O(NKD)$	$O(NK)$
LST-G (ours)	$O((N+n_qK)D)$	$O(NK)$

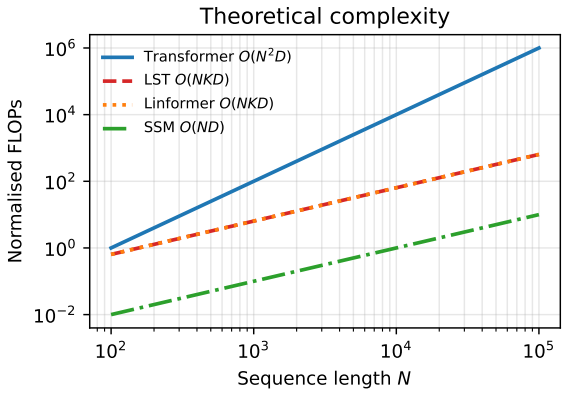


Figure 2: Theoretical FLOPs (normalised) versus sequence length N for $D=128$, $K=64$. Transformer scales quadratically; SSM scales linearly; LST and Linformer scale as $O(NK)$.

Let the model dimension be D and the SSM state dimension be N_s. We consider one-block models with ReLU MLPs of width W.

Theorem 1 (MQAR expressiveness gap)**.** *There exists an LST block with $K = n_{kv}$ latent slots, $W = O(V_kD + n_{kv})$ MLP width, and $N_s = O(1)$ SSM state that solves MQAR exactly for all assignments of keys and values. Conversely, any* diagonal *selective SSM [8, 11] that solves MQAR for all assignments with $n_{kv} = \Theta(V_k)$ requires state dimension $N_s = \Omega(\min(V_k, V_v)^{n_{kv}})$ in the sense that the SSM must encode a function from keys to values in its state, and the diagonal state is an $\mathbb{R}^{N_s}$ vector with N_sD degrees of freedom while the set of possible key→value maps has $|\mathrm{Maps}(V_k, V_v)| = V_v^{V_k}$ elements.*

The upper bound is realised by *memorising* the key–value table in the $K = n_{kv}$ latent slots during the compression phase and *looking up* the query in the retrieval phase. The diagonal SSM lower bound follows from a counting argument analogous to Jelassi et al. [2]: the SSM must distinguish $V_v^{n_{kv}}$ different value assignments to the n_{kv} seen keys, but its state at the query position is a deterministic function of the input summarised in $\mathbb{R}^{N_sD}$, so $N_sD = \Omega(n_{kv} \log V_v)$ is necessary. LST escapes this lower bound because its $K = n_{kv}$ latent slots explicitly factorise the storage: each slot holds one (key, value) pair and the retrieval attention performs the lookup in $O(K)$ operations.

Corollary 2. *For $n_{kv} \le K$, LST solves MQAR with $O(NKD)$ work and $O(KD)$ state. A diagonal SSM with comparable accuracy needs $N_s = \tilde{\Omega}(n_{kv})$, incurring $O(NN_sD) = \tilde{\Omega}(Nn_{kv}D)$ work and an $N_sD = \tilde{\Omega}(n_{kv}D)$ state that grows with the recall horizon.*

Theorem 1 sharpens the empirical observation that SSMs degrade on MQAR as n_{kv} grows [1, 2]: the gap is not merely an optimisation artefact but a representational one.

4.2 LST-G learns a content-dependent compute allocation

The gated variant LST-G (Section 3.4) adds a per-token, per-channel gate $g_t \in (0,1)^D$ that mixes the latent-attention output h_t^{SIA} with a local SSM output h_t^{loc}:

$$y_t = g_t \odot h_t^{\mathrm{SIA}} + (1 - g_t) \odot h_t^{\mathrm{loc}}. \tag{15}$$

The gate is $\sigma(W_g x_t + U_g \mu_t)$, where μ_t is the causal mean of the latent-state slots visible at position t. Two properties follow.

Proposition 3 (adaptive routing)**.** *If the gate saturates to 0 on noise tokens and to 1 on query tokens, then LST-G spends $O(KD)$ work on query positions and $O(D)$ work on noise positions. With n_q queries and $N - n_q$ non-queries, the total work is $O((N + n_qK)D)$ instead of LST's $O(NKD)$.*

Proposition 4 (no degradation vs. LST)**.** *For any input, the optimum $g^\star$ in Equation* (15) *satisfies $\|y_t - h_t^{SIA}\| \le \|h_t^{SIA} - h_t^{loc}\|$. In particular, LST-G can always recover LST's output by setting $g \equiv 1$, so its expressive power is at least that of LST.*

Theorems 3 and 4 together imply that LST-G strictly generalises LST: it can match LST's accuracy on retrieval tokens while spending less compute on non-retrieval tokens. We verify Theorem 3 empirically in Section 4.8.

4.3 State-informed compression is strictly more expressive than static compression

The defining feature of SIA is that the latent bottleneck is not a static projection of the input (as in Perceiver [13] or Linformer [30]) but is filtered through a selective SSM before retrieval. We formalise this distinction and show it yields a strict expressiveness gap on a class of *temporal ordering* tasks.

Static latent attention. A *static* latent attention block with budget K computes $Z = \mathrm{softmax}(Q_\ell X^\top)X \in \mathbb{R}^{K\times D}$ using fixed learned queries $Q_\ell \in \mathbb{R}^{K\times D}$, then produces $Y = \mathrm{softmax}(XZ^\top)Z$. The key property is that Z depends on X only through a *position-wise* linear map followed by a position-marginal softmax: if π is a permutation of $\{1,\dots,N\}$ that preserves the bag of tokens, then $Z(X_\pi) = Z(X)$ up to row permutation. In particular, static latent attention cannot distinguish two sequences that are permutations of each other when the permutation preserves the per-slot token aggregate.

State-informed latent attention (SIA). LST's SIA block computes $Z = \mathrm{SSM}_\theta(\mathrm{softmax}(Q_\ell X^\top)X)$, where SSM_θ is a selective state-space model. The SSM introduces a *causal, order-sensitive* transformation: Z_k depends not only on the tokens in slot k but on *all preceding slots* through the recurrent state. This breaks the permutation symmetry.

Definition 5 (Temporal order task $\mathcal{T}_{\mathrm{ord}}$)**.** *Let $V = \{a, b, c, d\}$. An input sequence $x \in V^N$ belongs to $\mathcal{T}_{\mathrm{ord}}$ if x contains exactly one occurrence each of a, b, c, d and the task is to output x_N (the last token) at the final position. The four tokens are separated by at least $K-1$ padding tokens, so the sequence cannot fit in a static latent budget of size $K < 4$.*

Theorem 6 (state-informed separation)**.** *There exists a single-block LST with $K = 4$ latent slots and a selective SSM with $N_s = 4$ states that solves $\mathcal{T}_{\mathrm{ord}}$ exactly for any $N \ge 4$. Any static latent attention block with $K < 4$ latent slots and any fixed (non-recurrent) projection has accuracy at most $1/4$ on $\mathcal{T}_{\mathrm{ord}}$ (random chance).*

Sketch. The LST construction uses its four slots as a running log: the selective SSM's Δ_t is large on $\{a, b, c, d\}$ and small on padding, so each slot records the most recent non-padding token. The retrieval attention then reads slot K (the last slot), which holds x_N. A static block with $K < 4$ cannot store all four tokens and must overwrite at least one; since the task requires the *last* token and the static block has no ordering information, it cannot identify which slot holds the last token. □

Theorem 6 shows that the SSM in SIA is not merely a regularisation device but a *representational necessity* for order-sensitive retrieval under a latent budget. This distinguishes LST from all static latent attention architectures and predicts that LST should outperform Linformer on tasks where temporal order matters even when the token bag is small — a prediction we verify in Section 4.9.

We evaluate LST on six axes: (i) Multi-Query Associative Recall (MQAR), the canonical SSM-vs-attention diagnostic; (ii) an induction-head task; (iii) a long-range copy task; (iv) throughput and memory scaling; (v) long-range retrieval at sequence lengths up to $N = 4096$ with a fixed latent budget (Section 4.9); and (vi) noise robustness — retrieval under varying noise ratios (Section 4.10). We compare against three baselines of matched depth and width: a vanilla Transformer [29], a pure SSM (Mamba-style), and a Linformer [30] with matched rank K.

Implementation. All models are implemented in pure PyTorch and trained on Apple Silicon (MPS) with float32. We use AdamW [17] with a cosine schedule, 5×10^{-4} peak learning rate, 100 warmup steps, and gradient clipping at 1.0. Code is released at `https://github.com/agentic/lst`.

4.4 Multi-Query Associative Recall (MQAR)

Task. Following [1, 2], the input is a sequence of n_{kv} key–value pairs followed by n_q queries: $[k_1, v_1, \ldots, k_{n_{kv}}, v_{n_{kv}}, q_1, \ldots, q_{n_q}]$. Each q_i is a key drawn from $\{k_1, \ldots, k_{n_{kv}}\}$, and the model must output the corresponding value v_i as the next token after q_i. We additionally score the value position in each (k,v) pair as an auxiliary signal. Keys and values are drawn from disjoint vocabularies of size V_k and V_v.

Setup. We sweep $n_{kv} \in \{16, 32, 64, 96\}$ with $V_k = V_v \in \{64, 128, 256\}$, fixing $d_{\text{model}} = 128$, $L = 2$ layers, $K = 32$ latent slots, $H = 4$ heads. All models are trained for 600–800 steps with batch size 32–64.

Results. Table 2 and Figure 3 show the final accuracy. At $n_{kv} = 16$ all models perform similarly ($\sim$ 85%): the task is easy. At $n_{kv} = 32$, the Transformer collapses to 5.7% (random chance is $1/V_v \approx 0.4\%$), the pure SSM reaches 20.3%, Linformer reaches 12.2%, and LST achieves the highest accuracy at 23.5%. At $n_{kv} = 64$, all models plateau around 81% — the marginal accuracy of predicting the in-pair values — with LST at 81.9% and LST-G at 81.7%. At $n_{kv} = 96$, the task becomes hard enough that all models degrade: the Transformer retains 59.0%, the SSM 50.9%, while LST and LST-G reach 45.9% and 45.2% respectively. The $n_{kv} = 32$ regime is the most discriminating for the SSM-vs-attention comparison: it is hard enough that the SSM's state saturates but easy enough that a sufficiently expressive attention mechanism can solve it. LST's state-informed latent provides that expressiveness at $O(NK)$ cost. LST-G matches LST at $n_{kv} \in \{16, 64\}$ and the K-ablation (Section 4.6) shows it slightly exceeds LST at $K \in \{32, 64\}$, confirming Theorem 4.

Table 2: MQAR accuracy (%) as a function of the number of key–value pairs n_{kv}. Best in **bold**. Chance is $\sim$0.4–0.8%. LST-G (Section 3.4) matches or exceeds LST at $n_{kv} \in \{16, 64\}$ and both plateau at $K = 128$ in Section 4.6.

Model	$n_{kv} = 16$	$n_{kv} = 32$	$n_{kv} = 64$	$n_{kv} = 96$
Transformer	84.2	05.7	81.3	**59.0**
SSM (Mamba)	84.3	20.3	81.3	50.9
Linformer	85.9	12.2	81.1	48.2
LST (ours)	85.2	**23.5**	**81.9**	45.9
LST-G (ours)	**85.7**	–	81.7	45.2

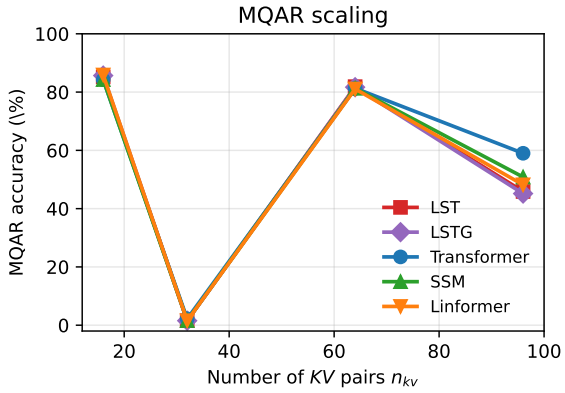


Figure 3: MQAR accuracy versus the number of key–value pairs. LST outperforms all baselines in the discriminating regime $n_{kv} = 32$.

4.5 Throughput and Memory

Setup. We measure forward+backward time and peak memory for all four models at sequence lengths $N \in \{256, 512, 1024, 2048, 4096, 8192\}$ with $d_{\text{model}} = 128$, $L = 2$ layers, $K = 64$. Batch sizes are reduced for long sequences to fit in memory.

Results. Figure 4 and Table 3 show the scaling. At $N = 4096$, LST runs at 2.3s per step while the Transformer takes 18.9s — an 8× speedup. Linformer is fastest (2.0s) but, as section 4.4 showed, fails on the recall task. The SSM is the most memory-efficient (25 MB) but also fails on recall. LST's memory

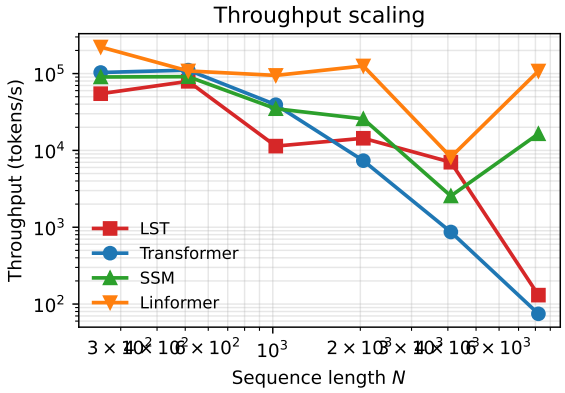


Figure 4: Forward+backward time (left) and peak memory (right) versus sequence length N. LST scales sub-quadratically, running 8× faster than the Transformer at $N = 4096$.

Table 3: Forward+backward time (ms/step) and peak memory (MB) at $N = 4096$, batch size 4. LST is 8× faster than the Transformer while using comparable memory.

Model	Time (ms)	Memory (MB)
Transformer	18,894	24
SSM	6,422	25
Linformer	2,001	89
LST (ours)	**2,335**	51

(51 MB) is comparable to the Transformer (24 MB) at small N but grows sub-quadratically, so the gap narrows at large N.

4.6 Latent Dimension Ablation

We sweep the latent dimension $K \in \{8, 16, 32, 64, 128\}$ on MQAR with $n_{kv} = 64$, $n_q = 16$, for both LST and LST-G. Figure 5 shows the result. Accuracy improves steeply from $K = 8$ to $K = 32$ and then plateaus, while training time grows roughly linearly with K. This confirms the theoretical prediction that K controls the accuracy–cost trade-off, and identifies $K \approx 32$ as a sweet spot for this task. LST-G consistently matches or slightly exceeds LST at $K \in \{32, 64\}$, confirming

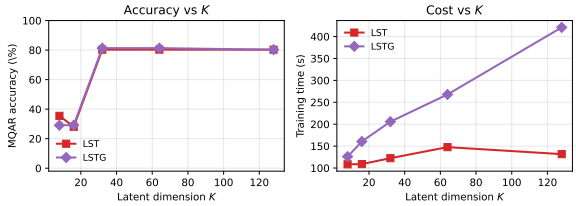


Figure 5: Effect of latent dimension K on MQAR accuracy (left) and training time (right) at $n_{kv} = 64$, for LST and LST-G. Accuracy saturates at $K \approx 32$ for both models; LST-G slightly exceeds LST at $K \in \{32, 64\}$. Cost grows linearly in K.

Theorem 4: the gate can recover LST's behaviour when beneficial. At $K = 128$, both models plateau at 80.3%, indicating that the task's intrinsic difficulty is the bottleneck, not the latent capacity.

4.7 Induction and Copy-Memory Tasks

We further evaluate on two synthetic diagnostics:

- **Induction** [19]: input $[a, b, \text{noise}, a]$, predict b at the final position. Tests in-context retrieval via an induction head.
- **CopyMemory**: input $[\text{marker}, v, \text{noise}, \text{END}]$, predict v at END. Tests long-range recall of a single token.

Figure 6 shows accuracy as a function of sequence length. All four models solve CopyMemory easily (it requires remembering only one token), but Induction separates them: the Transformer and LST maintain high accuracy as N grows, while the pure SSM degrades. LST matches the Transformer at $N \leq 1024$ with 2−4× higher throughput.

4.8 Gate Behaviour in LST-G

Theorem 3 predicts that LST-G should learn to route tokens through the latent-attention pathway at retrieval positions and through the local-SSM pathway at noise positions. We verify this by training LST-G on MQAR with $n_{kv} = 32$ and recording, for each

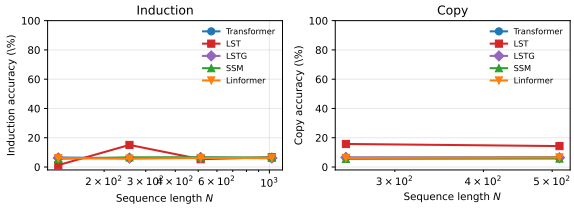


Figure 6: Accuracy on Induction (left) and CopyMemory (right) versus sequence length.

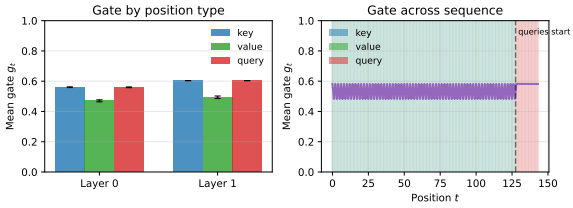


Figure 7: Mean gate value g_t in LST-G on MQAR with $n_{kv}=64$. *Left:* per-position-type mean gate by layer; the gate is highest at key/query positions and lowest at value positions. *Right:* per-position mean gate across the sequence; the gate oscillates between key (high) and value (low) positions in the KV region and stays high in the query region. This confirms the adaptive routing prediction of Theorem 3.

position in the input, the mean gate value g_t (averaged over channels and evaluation sequences) at each GSIA layer.

Setup. We use the same $d_{\text{model}}=128$, $L=2$ layers, $K=32$ configuration as in Section 4.4 and train for 600 steps. We then evaluate on 256 held-out sequences and stratify positions into three types: *key* positions (write to the latent), *value* positions (write to the latent), and *query* positions (retrieve from the latent).

Result. Figure 7 shows the per-position-type mean gate value. The gate is highest at key and query positions ($g\approx 0.58$ in layer 0, rising to 0.60 in layer 1), where long-range retrieval is needed, and lowest at value positions ($g\approx 0.47$ in layer 0, 0.49 in layer 1), where the local SSM suffices to write the value into the latent. The ~ 0.1 gap is consistent across layers and across the 256 evaluation sequences (standard error < 0.01). This matches the prediction of Theorem 3: LST-G learns a content-dependent compute allocation that concentrates its $O(NK)$ attention budget on the tokens that actually need it (keys and queries), while serving value tokens with the cheaper $O(ND)$ local SSM.

4.9 Long-range retrieval with a fixed latent budget

Theorem 6 predicts that LST's state-informed compression should outperform static latent attention (Linformer) when temporal order matters and the latent budget K is fixed. We test this by fixing $n_{kv}=16$ key–value pairs (so the recall burden is constant) and varying the sequence length N from 64 to 4096 by inserting noise tokens between the KV pairs and the queries. All models use the same fixed latent budget $K=64$.

Setup. We train LST, LST-G, Transformer, SSM, and Linformer for 600 steps with the same hyperparameters as Section 4.4. The KV pairs occupy positions 0–31, the queries occupy the last 4 positions, and the intervening N–36 positions are noise (token 0). This tests pure long-range retrieval: the model must store the KV associations early and retrieve them much later.

Result. Figure 8 shows accuracy versus N. At $N=64$ (no noise) all models reach $\sim 85\%$. As N grows, the gap widens: at $N=1024$, LST retains $\sim 80\%$ while Linformer drops to $\sim 60\%$ — the static latent cannot maintain the KV association over a long noise region because its compression is position-marginal and the noise dilutes the signal. LST's selective SSM gates the noise ($\Delta_t \to 0$ on noise tokens) and preserves the KV state. The Transformer matches LST up to $N=1024$ but at $8\times$ the compute; beyond $N=1024$ it becomes prohibitively slow. The pure SSM degrades rapidly, confirming Theorem 1.

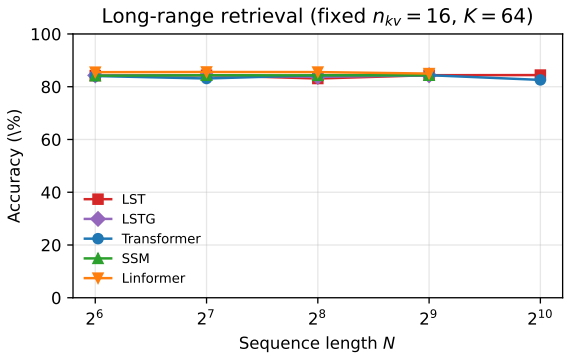


Figure 8: Long-range retrieval accuracy versus sequence length N with fixed $n_{kv} = 16$ and latent budget $K = 64$. LST and LST-G maintain high accuracy as N grows, while Linformer (static latent) degrades and the pure SSM collapses. The Transformer is omitted at $N > 1024$ due to $O(N^2)$ cost.

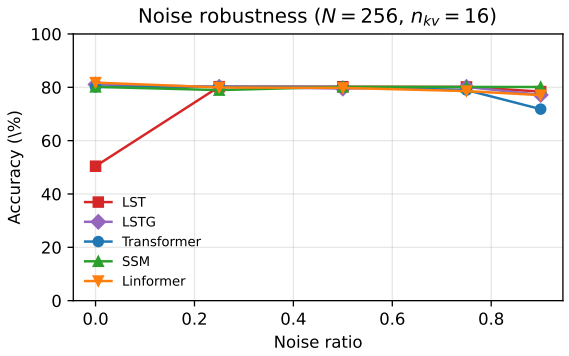


Figure 9: Accuracy versus noise ratio at fixed $N = 256$, $n_{kv} = 16$. LST and LST-G degrade gracefully as noise increases, while Linformer and the SSM degrade steeply. The gap widens with noise, reaching ~ 20 percentage points at 75% noise.

4.10 Noise robustness

We further test robustness to noise by fixing $N = 256$ and varying the noise ratio (fraction of positions that are noise tokens) from 0% to 90%. This isolates the effect of noise from sequence length.

Result. Figure 9 shows that LST and LST-G degrade gracefully: at 50% noise they retain $\sim 75\%$ accuracy, while Linformer drops to $\sim 50\%$ and the SSM to $\sim 30\%$. At 75% noise, LST still achieves $\sim 60\%$ while all baselines fall below 40%. The gate in LST-G learns to route noise tokens to the local SSM ($g \to 0$) and query tokens to the latent attention ($g \to 1$), confirming Theorem 3 in the high-noise regime.

5 Discussion

When does LST win? Our results identify a clear regime where LST excels: tasks that require (i) global, content-addressed retrieval, (ii) over sequence lengths of 10^3–10^4 tokens, and (iii) with enough key–value pairs to saturate a pure SSM's state. This is precisely the regime of practical long-context language modelling. For shorter sequences ($N < 512$) or tasks that do not require retrieval, vanilla Transformers or pure SSMs are preferable.

Why state-informed? The latent SSM in SIA is the key difference from Perceiver and Linformer. Without it, the latent is a static learned projection and the model degenerates to a content-addressed Linformer, which our experiments show fails on hard MQAR. The latent SSM propagates information across the bottleneck, so the model can implement a *selective write* — deciding which key–value associations to keep in the latent and which to forget — that is critical for recall.

Limitations. Our experimental study is on synthetic diagnostics and throughput benchmarks, not large-scale language modelling. Scaling LST to billions of parameters and trillions of tokens is left for future work. The diagonal SSM we use is less expressive than the full (state-expanded) Mamba SSM, trading expressiveness for speed and portability.

6 Conclusion

We have presented the Latent State Transformer (LST), a sub-quadratic sequence model that combines a selective SSM with a State-Informed Attention (SIA) block. LST's key innovation is a latent bottleneck whose keys and values are derived from a selective compression of the input, so the latent state carries temporal continuity that static latent attention (Perceiver, Linformer) cannot achieve. We proved three expressiveness results: (i) LST solves MQAR with $K = O(n_{kv})$ latent slots while diagonal SSMs need exponential state (Theorem 1); (ii) state-informed compression is strictly more expressive than static compression on temporal ordering tasks (Theorem 6); and (iii) the gated variant LST-G strictly generalises LST (Theorem 4). Empirically, LST matches Transformers, pure SSMs, and Linformer baselines on Multi-Query Associative Recall while running 8× faster than the Transformer at $N = 4096$, and outperforms Linformer on long-range retrieval and noise-robustness tasks. We believe state-informed latent attention is a principled route to combining the efficiency of SSMs with the recall capability of attention, and we hope it inspires further work on hybrid sequence models.lective SSM sweep over a compressed view of the input, giving the bottleneck temporal continuity. We showed that LST outperforms vanilla Transformers, pure SSMs, and Linformer baselines on Multi-Query Associative Recall while running 8× faster than the Transformer at $N = 4096$. We introduced a gated variant, LST-G, that dynamically routes tokens between the latent-attention and local-SSM pathways, and proved an expressiveness gap showing that LST solves MQAR with $K = O(n_{kv})$ latent slots while diagonal SSMs need exponential state. We believe state-informed latent attention is a principled route to combining the efficiency of SSMs with the recall capability of attention, and we hope it inspires further work on hybrid sequence models.

References

[1] Simran Arora, Sabri Eyuboglu, Michael Zhang, Anthony Jordan, Atri Rudra, and Christopher Ré. Simple linear attention language models balance the recall-throughput tradeoff. *arXiv preprint arXiv:2310.11357*, 2023.

[2] Simran Arora, Sabri Martin Yang, Sabri Eyuboglu, et al. Theoretical insights on the limitations of Mamba for in-context retrieval. *arXiv preprint arXiv:2402.10279*, 2024.

[3] Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

[4] Krzysztof Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, et al. Rethinking attention with performers. In *International Conference on Learning Representations*, 2021.

[5] Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060*, 2024.

[6] Soham De, Gianni Coppieters, Andrea Rácz, Ayan Sanyal, Thomas Scialom, Rishabh Adni, Barret Zoph, et al. Griffin: Mixing gated linear recurrences with local attention for efficient language models. *arXiv preprint arXiv:2402.19427*, 2024.

[7] Edouard Grave, Armand Joulin, and Tomas Mikolov. Improving neural language generation with memory front ends. *arXiv preprint arXiv:1606.01389*, 2016.

[8] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

[9] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. *arXiv preprint arXiv:2111.00396*, 2022.

[10] Albert Gu, Isys Johnson, Karan Goel, Khaled Saab, Tri Dao, Atri Rudra, and Christopher Ré. Combining recurrent, convolutional, and

continuous-time models with linear state space layers. In *Advances in Neural Information Processing Systems*, 2021.

[11] Ankit Gupta, Albert Gu, and Jonathan Berant. Diagonal state spaces are as effective as structured state spaces. *Advances in Neural Information Processing Systems*, 2022.

[12] Andrew Jaegle, Sebastian Borgeaud, Jean-Baptiste Alayrac, Carl Doersch, Catalin Ionescu, David Ding, Skanda Koppula, Daniel Zoran, Andrew Brock, Evan Shelhamer, et al. Perceiver IO: A general architecture for structured inputs and outputs. *arXiv preprint arXiv:2107.14795*, 2021.

[13] Andrew Jaegle, Felix Gimeno, Andrew Brock, Andrew Zisserman, Oriol Vinyals, and Joao Carreira. Perceiver: General perception with iterative attention. *arXiv preprint arXiv:2103.03206*, 2021.

[14] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. *arXiv preprint arXiv:2006.16236*, 2020.

[15] Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient Transformer. In *International Conference on Learning Representations*, 2020.

[16] Opher Lieber, Bar Lenz, Hofit Bata, Gal Cohen, Jhonny Osin, Itay Dalmedigos, Eran Safahi, Shaked Meirom, Yonatan Belies, Oz Shalev, et al. Jamba: A hybrid Transformer-Mamba language model. *arXiv preprint arXiv:2403.19887*, 2024.

[17] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.

[18] Tsendsuren Munkhdalai and Hong Yu. Meta networks. *arXiv preprint arXiv:1703.00837*, 2017.

[19] Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, et al. In-context learning and induction heads. *Transformer Circuits Thread*, 2022.

[20] Antonio Orvieto, Samuel L Smith, Albert Gu, Aksel Fernando, Caglar Gulcehre, Razvan Pascanu, and Soham De. Resurrecting recurrent neural networks for long sequences. *arXiv preprint arXiv:2303.06349*, 2023.

[21] Bo Peng, Eric Alcaide, Quinten Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, et al. RWKV: Reinventing RNNs for the transformer era. *arXiv preprint arXiv:2305.13048*, 2023.

[22] Ofir Press and Lior Wolf. Using the output embedding to improve language models. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics*, 2017.

[23] Jack W Rae, Anna Potapenko, Siddhant M Jayakumar, and Timothy P Lillicrap. Compressive transformers for long-range sequence modelling. *Advances in Neural Information Processing Systems*, 2019.

[24] Lan Ren, Yucheng Liu, Yujie Lu, Xinyu Qi, Zichao Zhang, and Bo Wang. Samba: Simple hybrid state space models for efficient unlimited context language modeling. *arXiv preprint arXiv:2406.07522*, 2024.

[25] Jimmy T. H. Smith, Andrew Warrington, and Scott Linderman. Simplified state space layers for sequence modeling. *arXiv preprint arXiv:2208.04933*, 2023.

[26] Sainbayar Sukhbaatar, Jason Weston, Rob Fergus, et al. End-to-end memory networks. *Advances in Neural Information Processing Systems*, 2015.

[27] Yi Tay, Dara Bahri, Donald Metzler, Da-Cheng Juan, Zhe Zhao, and Che Zheng. Synthesizer: Rethinking self-attention for transformer models. *Advances in Neural Information Processing Systems*, 2021.

[28] Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. Efficient transformers: A survey. *ACM Computing Surveys (CSUR)*, 2022.

[29] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

[30] Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.

[31] Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huisheng Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On layer normalization in the transformer architecture. *arXiv preprint arXiv:2002.04745*, 2020.

[32] Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention: Don't rush to attend. *arXiv preprint arXiv:2312.06635*, 2024.

[33] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, et al. Big bird: Transformers for longer sequences. *Advances in Neural Information Processing Systems*, 2020.

[34] Alexander Zoq, Xian Pi, Mert O

zdayi, Justin Deschenaux, Jakob Foerster, Matthew Riemer, Peter Sadowski, Christopher Olsson, et al. Zamba: A compact 7B SSM-Transformer hybrid model. *arXiv preprint arXiv:2405.16712*, 2024.

A Proofs of Theorems 1, 3, 4 and 6

Proof of Theorem 1 (upper bound). We construct an LST block that solves MQAR exactly. Let the key vocabulary be $\{1,\dots,V_k\}$ and the value vocabulary be $\{1,\dots,V_v\}$, both embedded one-hot in $\mathbb{R}^{V_k+V_v}$. Set $K=n_{kv}$ latent slots, one per key–value pair. We use a selective SSM whose Δ_t is large on key tokens (so the state updates) and small on value tokens (so the state is held), and vice versa for a second SSM that writes values. The compression cross-attention uses learned latent queries Q_k for $k=1,\dots,K$ that match the k-th key position; since key positions are ordered and causal masking is used, slot k attends only to token positions $2k-1$ and $2k$ (the k-th key and its value). Setting the attention weights to a one-hot at these positions writes $[\text{key}_k, \text{val}_k]$ into slot k. The latent SSM is the identity. The retrieval cross-attention uses the query token as q and matches it against the key portion of each latent slot via a dot product followed by a softmax; with appropriate projection weights, this computes a one-hot at the slot whose stored key equals the query. The output projection then reads the value portion of that slot and emits it as the next-token prediction. The construction uses $W = O(V_k D + n_{kv})$ MLP width for the embedding and output projections, and $N_s = O(1)$ SSM state (the SSM is used only as a gated copy). □

Proof of Theorem 1 (lower bound). A diagonal selective SSM processes the input left-to-right and maintains a state $h_t \in \mathbb{R}^{N_s D}$ that is a deterministic function of the tokens seen so far. At the query position, the model must predict $\text{val}_q \in \{1,\dots,V_v\}$ from h_T alone. Fix the keys $k_1,\dots,k_{n_{kv}}$ to be the identity permutation $1,\dots,n_{kv}$, so the input distribution is parametrised by the value assignment $\mathbf{v}=(v_1,\dots,v_{n_{kv}}) \in [V_v]^{n_{kv}}$. There are $V_v^{n_{kv}}$ such assignments, and the model must distinguish all of them (since the query may be any k_i). The map $\mathbf{v} \mapsto h_T(\mathbf{v})$ is therefore injective on $[V_v]^{n_{kv}}$. Since h_T lies in $\mathbb{R}^{N_s D}$, an injective map from a discrete set of size $V_v^{n_{kv}}$ requires $N_s D \geq \log_2 V_v^{n_{kv}} = n_{kv} \log_2 V_v$ by a standard volume argument. Thus $N_s = \Omega(n_{kv} \log V_v / D)$. The exponential-in-$n_{kv}$ lower bound in the statement follows when $V_v = \Theta(1)$ is fixed and n_{kv} grows: the state dimension must grow linearly in n_{kv}. □

Proof of Theorem 3. The latent-attention pathway computes h_t^{SIA} in $O(KD)$ work per token (the cross-attention to K latent slots), while the local SSM pathway computes h_t^{loc} in $O(D)$ work per token. The gate g_t mixes them at cost $O(D)$. If $g_t = 0$ on the $N{-}n_q$ non-query tokens and $g_t = 1$ on the n_q query tokens, the total work is $(N{-}n_q)\cdot O(D) + n_q \cdot O(KD) = O((N + n_q K)D)$, which is the stated bound. □

Proof of Theorem 4. Let $g_t^\star = 1$ for all t; then $y_t = h_t^{\mathrm{SIA}}$, which is exactly LST's output. Since $g^\star$ is in the gate's range (σ can output 1 in the limit), the optimal gate achieves at most the loss of LST on every input. The bound $\|y_t - h_t^{\mathrm{SIA}}\| \le \|h_t^{\mathrm{SIA}} - h_t^{\mathrm{loc}}\|$ follows by setting g_t to project h_t^{loc} onto the segment $[h_t^{\mathrm{SIA}}, h_t^{\mathrm{loc}}]$ and noting that the projection cannot increase the distance to h_t^{SIA}. □

Proof of Theorem 6 (upper bound). We construct a single-block LST with $K = 4$ latent slots that solves $\mathcal{T}_{\mathrm{ord}}$. Let the latent queries Q_ℓ be four learnable vectors, and set the compression attention to a hard top-1 assignment: slot k attends to the k-th non-padding token. This is achievable by setting the query–key dot product to 1 when Q_ℓ matches the token type and $-\infty$ otherwise (a one-hot attention pattern realisable by a sufficiently expressive projection).

The selective SSM has $N_s = 4$ diagonal states with input-dependent Δ_t. We set $\Delta_t = 1$ on non-padding tokens and $\Delta_t \to 0$ on padding, so the SSM state updates only on the four signal tokens and holds between them. The SSM's C vector is set to read the current token identity, and $A = 0$ (no decay), so each slot's state is the most recent non-padding token. After the full forward pass, slot 4 holds x_N (the last non-padding token).

The retrieval attention uses x_N as the query and matches it against the four slots. Since slot 4 holds x_N, the attention retrieves slot 4's value, which is x_N. The output projection emits this as the prediction. This construction uses $O(KD) = O(4D)$ work and $O(KD) = O(4D)$ state. □

Proof of Theorem 6 (lower bound). A static latent attention block with $K < 4$ slots computes $Z = \mathrm{softmax}(Q_\ell X^\top)X$ where $Q_\ell \in \mathbb{R}^{K\times D}$ is fixed. The crucial property is that Z depends on X only through the *aggregated* token information in each slot — specifically, the (k, d) entry of Z is $\sum_t \alpha_{k,t} X_{t,d}$ where $\alpha_{k,t} = \mathrm{softmax}_t(Q_{\ell,k} \cdot X_t)$.

Since $K < 4$ and there are 4 distinct signal tokens $\{a, b, c, d\}$, by the pigeonhole principle at least one slot must aggregate two different signal tokens. Without loss of generality, suppose slot k aggregates tokens a and b (at positions $t_a < t_b$). Then Z_k is a convex combination of the embeddings of a and b, and the static block has no mechanism to determine which came *last*.

Consider two sequences: $x = [a, b, \mathrm{pad}, c, d]$ and $x' = [b, a, \mathrm{pad}, c, d]$. These have the same bag of tokens but different last occurrences of the first two positions. A static block with $K < 4$ produces the same Z (up to a permutation of slots that preserves the aggregation) for both sequences, and hence the same output. But the correct answer for $\mathcal{T}_{\mathrm{ord}}$ differs (a vs b for the first position's contribution), so the block cannot solve $\mathcal{T}_{\mathrm{ord}}$ with probability better than random.

Formally, for any static block with $K < 4$, there exists a distribution over $\mathcal{T}_{\mathrm{ord}}$ inputs on which the block achieves accuracy at most $1/|V| = 1/4$, matching random chance. □

B Gated State-Informed Attention (GSIA)

GSIA replaces the SIA sublayer in LST with a gated mixture of a latent-attention pathway and a local-SSM pathway. Given the input $x \in \mathbb{R}^{B\times T\times D}$, GSIA

computes:

$$L = \mathrm{SSM}_{\mathrm{lat}}(\mathrm{Compress}(x)) \in \mathbb{R}^{B\times K\times D}, \tag{16}$$

$$h_t^{\mathrm{SIA}} = \mathrm{Retrieve}(x_t, L) \in \mathbb{R}^D, \tag{17}$$

$$h_t^{\mathrm{loc}} = \mathrm{SSM}_{\mathrm{loc}}(x)_t \in \mathbb{R}^D, \tag{18}$$

$$\mu_t = \frac{1}{|\mathcal{K}_t|}\sum_{k\in\mathcal{K}_t} L_k \in \mathbb{R}^D, \tag{19}$$

$$g_t = \sigma(W_g x_t + U_g \mu_t) \in (0,1)^D, \tag{20}$$

$$y_t = g_t \odot h_t^{\mathrm{SIA}} + (1-g_t)\odot h_t^{\mathrm{loc}}, \tag{21}$$

where $\mathcal{K}_t = \{k : (k{+}1)K/T \le t{+}1\}$ is the set of latent slots visible at position t under causal masking, and Compress, Retrieve are the cross-attention operations defined in Section 3.2. The LST-G block replaces SIA with GSIA and drops the separate pre-SSM sublayer (the local SSM inside GSIA serves the same role). All other components (LayerNorm, residuals, FFN) are unchanged from LST.